NegativeScreenDemo 技术文档

概述

本文档详细说明 Android 负一屏 WebView Demo 的实现原理、遇到的问题及解决方法，以及如何对接 ZTE 设备的负一屏系统。

一、项目结构

```
NegativeScreenDemo/
├── app/
│   ├── build.gradle                          ← 模块构建配置（含 viewBinding）
│   └── src/main/
│       ├── AndroidManifest.xml               ← 应用清单（注册 Activity/Service/Receiver）
│       ├── assets/
│       │   └── index.html                    ← H5 演示页面
│       ├── java/com/demo/negative/
│       │   ├── MainActivity.kt               ← 主界面
│       │   ├── NegativeScreenActivity.kt     ← 负一屏（全屏 WebView）
│       │   ├── provider/
│       │   │   └── NegativeScreenWidgetProvider.kt   ← 桌面小组件
│       │   └── service/
│       │       └── NegativeScreenService.kt          ← 后台服务（当前未使用）
│       └── res/
│           ├── layout/
│           │   ├── activity_main.xml
│           │   ├── activity_negative_screen.xml
│           │   └── widget_negative_screen.xml
│           ├── xml/
│           │   └── negative_screen_widget_info.xml
│           └── values/
│               └── styles.xml
├── build.gradle
├── settings.gradle
└── gradle.properties
```

二、遇到的问题及解决方法

问题 1：AndroidManifest 组件位置错误

错误信息：

```
error: unexpected element <receiver> found in <manifest>.
error: unexpected element <service> found in <manifest>.
error: unexpected element <activity> found in <manifest>.
```

原因： 组件（receiver、service、activity）放在了 `<application>` 标签外部。

解决方法： 所有组件必须嵌套在 `<application>` 内部：

```
<application>
    <!-- 正确：组件放在 application 内部 -->
    <receiver>...</receiver>
    <service>...</service>
    <activity>...</activity>
</application>
```

问题 2：Kotlin JVM 版本不兼容

错误信息：

```
Inconsistent JVM-target compatibility detected for tasks
'compileDebugJavaWithJavac' (1.8) and 'compileDebugKotlin' (17).
```

原因： Java 编译默认使用 1.8，Kotlin 使用 17，版本不一致。

解决方法： 在 `app/build.gradle` 中添加：

```
android {
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }
    kotlinOptions {
        jvmTarget = '17'
    }
}
```

问题 3：缺少 Kotlin 插件

错误信息：

```
Could not open settings generic class cache ... Unsupported class file major version 69
```

原因： 系统默认 Java 25 与 Gradle 8.4 不兼容，需要 Java 17。

解决方法：

1. Android Studio 设置 Gradle JDK 为 Java 17
2. 或在命令行执行：`export JAVA_HOME=/Users/len/Library/Java/JavaVirtualMachines/jbr-17.0.14/Contents/Home`

问题 4：Theme.AppCompat 主题缺失

错误信息：

```
java.lang.IllegalStateException: You need to use a Theme.AppCompat theme (or descendant) with t
```

原因： `MainActivity` 继承 `AppCompatActivity`，但未指定 AppCompat 主题。

解决方法：

1. 创建 `res/values/styles.xml`：

```
<style name="AppTheme" parent="Theme.AppCompat.Light.NoActionBar">
    <item name="colorPrimary">#667EEA</item>
    <item name="colorPrimaryDark">#764BA2</item>
    <item name="colorAccent">#667EEA</item>
</style>
```

2. 在 `AndroidManifest.xml` 的 `<application>` 中指定主题：

```
android:theme="@style/AppTheme"
```

问题 5：ClassNotFoundException

错误信息：

```
java.lang.ClassNotFoundException: Didn't find class "com.demo.negative.MainActivity"
```

原因： APK 安装包中未正确包含 Kotlin 编译的类文件。

解决方法：

1. 执行 `./gradlew clean`
2. 重新 `./gradlew assembleDebug`
3. 卸载旧 APK 后重新安装

问题 6：ViewBinding 未启用

错误信息：

```
Unresolved reference: ActivityNegativeScreenBinding
```

原因： `build.gradle` 中未启用 viewBinding。

解决方法： 在 `app/build.gradle` 添加：

```
android {
    buildFeatures {
        viewBinding true
    }
}
```

三、实现原理

架构图

```
┌──────────────────────────────────────────────────────────┐
│                          用户                            │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│                       MainActivity                       │
│                 (主界面，点击按钮打开负一屏)              │
└──────────────────────────┬───────────────────────────────┘
                           │ onClick
                           ▼
┌──────────────────────────────────────────────────────────┐
│                NegativeScreenActivity                    │
│                      (全屏 Activity)                     │
│   ┌──────────────────────────────────────────────┐       │
│   │              activity_negative_screen.xml    │       │
│   │   ┌──────────────────────────────────────┐   │       │
│   │   │                WebView               │   │       │
│   │   │  加载 assets/index.html (H5 页面)    │   │       │
│   │   └──────────────────────────────────────┘   │       │
│   └──────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────┘
```

核心组件说明

1. MainActivity（主界面）

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    // 点击"打开负一屏"按钮，启动全屏 Activity
    fun openNegativeScreen(view: View) {
        val intent = Intent(this, NegativeScreenActivity::class.java)
        startActivity(intent)
    }

    // 点击"停止"按钮，停止后台服务
    fun stopNegativeScreen(view: View) {
        stopService(Intent(this, NegativeScreenService::class.java))
    }
}
```

2. NegativeScreenActivity（全屏 WebView）

```
class NegativeScreenActivity : AppCompatActivity() {

    private lateinit var binding: ActivityNegativeScreenBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityNegativeScreenBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupWebView()
    }

    private fun setupWebView() {
        // 配置 WebView
        binding.webView.settings.apply {
            javaScriptEnabled = true           // 启用 JS
            domStorageEnabled = true           // 启用 DOM 存储
            useWideViewPort = true             // 支持响应式
            loadWithOverviewMode = true         // 适应屏幕宽度
            setSupportZoom(false)              // 禁用缩放
            mixedContentMode = WebSettings.MIXED_CONTENT_COMPATIBILITY_MODE  // HTTPS 支持
        }

        // 加载本地 H5 页面
        binding.webView.loadUrl("file:///android_asset/index.html")
    }
}
```

3. H5 页面（index.html）

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>智慧屏 Demo</title>
    <style>
        body {
            font-family: -apple-system, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            color: #fff;
        }
        /* 实时时钟 */
        #clock { font-size: 48px; font-weight: 300; }
    </style>
</head>
<body>
    <div id="clock">--:--</div>
    <script>
        // 每秒更新时钟
        function updateClock() {
            const now = new Date();
            document.getElementById('clock').textContent =
                String(now.getHours()).padStart(2,'0') + ':' +
                String(now.getMinutes()).padStart(2,'0');
        }
        setInterval(updateClock, 1000);
    </script>
</body>
</html>
```

四、目前展示方式

当前实现（全屏 Activity）

用户点击主界面「打开负一屏」按钮后，`NegativeScreenActivity` 以全屏方式展示 H5 页面。

特点：

- 简单直接，无需复杂配置
- 适合演示和快速验证
- 跨设备兼容性好

局限性：

- 不是真正的"桌面左滑"负一屏入口

- 无法在桌面直接左滑触发

H5 页面功能

功能	说明
实时时钟	每秒更新时间显示，格式 `HH:mm`
快捷功能卡片	展示快捷开关、天气、日程等信息
紫色渐变背景	渐变色主题 (#667EEA → #764BA2)
关闭按钮	右上角按钮，点击退出负一屏

五、如何接入 ZTE 设备的负一屏

重要说明

桌面左滑进入负一屏是**系统级功能**，普通 App 无法通过代码直接实现。这是 ZTE/中兴定制的 Launcher 功能，需要厂商配合。

接入步骤

第一步：联系 ZTE 获取 SDK

联系 ZTE 技术支持，获取以下材料：

1. **智慧屏 SDK**（如 `SmartScreenSDK` 或类似）
2. **签名授权**（特殊签名的 APK 才能注册负一屏入口）
3. **开发文档**（通常包含 `shortcut.xml`、`widget_info.xml` 配置规范）

第二步：配置负一屏组件

根据 ZTE 文档，通常需要：

1. **创建主控 APK**（主入口，管理智慧屏服务）
2. **创建智慧屏 APK**（具体功能实现）

典型配置：

```
<!-- 主控 APK AndroidManifest.xml -->
<receiver android:name=".SmartScreenReceiver">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.HOME" />
    </intent-filter>
    <meta-data
        android:name="android.app.shortcuts"
        android:resource="@xml/shortcuts" />
</receiver>
```

```
<!-- res/xml/shortcuts.xml -->
<shortcuts xmlns:android="http://schemas.android.com/apk/res/android">
    <shortcut android:shortcutId="smart_screen"
        android:enabled="true"
        android:icon="@drawable/ic_smart_screen"
        android:label="@string/smart_screen_label">
        <intent android:action="android.intent.action.MAIN"
            android:targetPackage="com.example.smart"
            android:targetClass=".SmartScreenActivity" />
    </shortcut>
</shortcuts>
```

第三步：安装并测试

按照 ZTE 文档指定的顺序安装 APK：

1. 先安装主控 APK
2. 再安装智慧屏 APK
3. 在桌面从左边缘向左滑动触发负一屏

常见问题

问题	解决方案
安装后左滑无反应	检查安装顺序，或需要重新启动设备
签名校验失败	联系 ZTE 获取正式签名授权
文档不完整	向 ZTE 技术支持索取完整 SDK

六、当前 Demo 的局限性与建议

局限性

1. **不是真正的系统负一屏**：当前 Demo 以 Activity 方式展示，无法实现桌面左滑触发
2. **跨设备兼容性**：不同厂商的负一屏实现方式不同，需要厂商 SDK 支持
3. **权限限制**：负一屏入口注册需要系统级权限，普通 App 无法自行实现

建议后续工作

1. **联系 ZTE 获取 SDK**：获取智慧屏 SDK 和签名授权
2. **参考厂商文档**：按照 ZTE 提供的文档配置负一屏入口
3. **双 APK 方案**：如果需要实现真正的负一屏，可能需要主控 + 智慧屏两个 APK

七、总结

项目	说明
当前 Demo	展示 WebView 加载 H5 页面效果，点击按钮全屏查看
真正负一屏	需要 ZTE 系统级支持，无法通过普通 App 实现
核心代码	`NegativeScreenActivity` + `assets/index.html`
技术栈	Kotlin + ViewBinding + WebView + H5

附录：关键代码片段

WebView 配置

```
webView.settings.apply {
    javaScriptEnabled = true           // 启用 JavaScript
    domStorageEnabled = true            // 启用 DOM 存储 (LocalStorage)
    databaseEnabled = true              // 启用数据库
    cacheMode = WebSettings.LOAD_DEFAULT // 缓存模式
    useWideViewPort = true              // 支持响应式布局
    loadWithOverviewMode = true         // 适应屏幕宽度
    setSupportZoom(false)               // 禁用缩放
    builtInZoomControls = false         // 隐藏内置缩放控件
    mixedContentMode = WebSettings.MIXED_CONTENT_COMPATIBILITY_MODE  // HTTPS 兼容
    allowFileAccess = true              // 允许访问文件
    allowContentAccess = true           // 允许访问内容
}
```

加载本地 H5

```
// 加载 assets 目录下的文件
webView.loadUrl("file:///android_asset/index.html")

// 加载远程 URL
webView.loadUrl("https://your-server.com/negative-screen/index.html")
```

Android 与 H5 交互

```
// Android 调用 H5
webView.evaluateJavascript("javascript:updateStatus('hello')", null)

// H5 调用 Android (通过 JS 接口)
webView.addJavascriptInterface(object {
    @JavascriptInterface
    fun sendToAndroid(message: String) {
        Log.d("WebView", "收到 H5 消息: $message")
    }
}, "AndroidInterface")
```

文档版本：v1.0 最后更新：2026-04-29